

CS376 - HW1 Report

학번 비공개 Sim Hyeonseong

[Task 1]

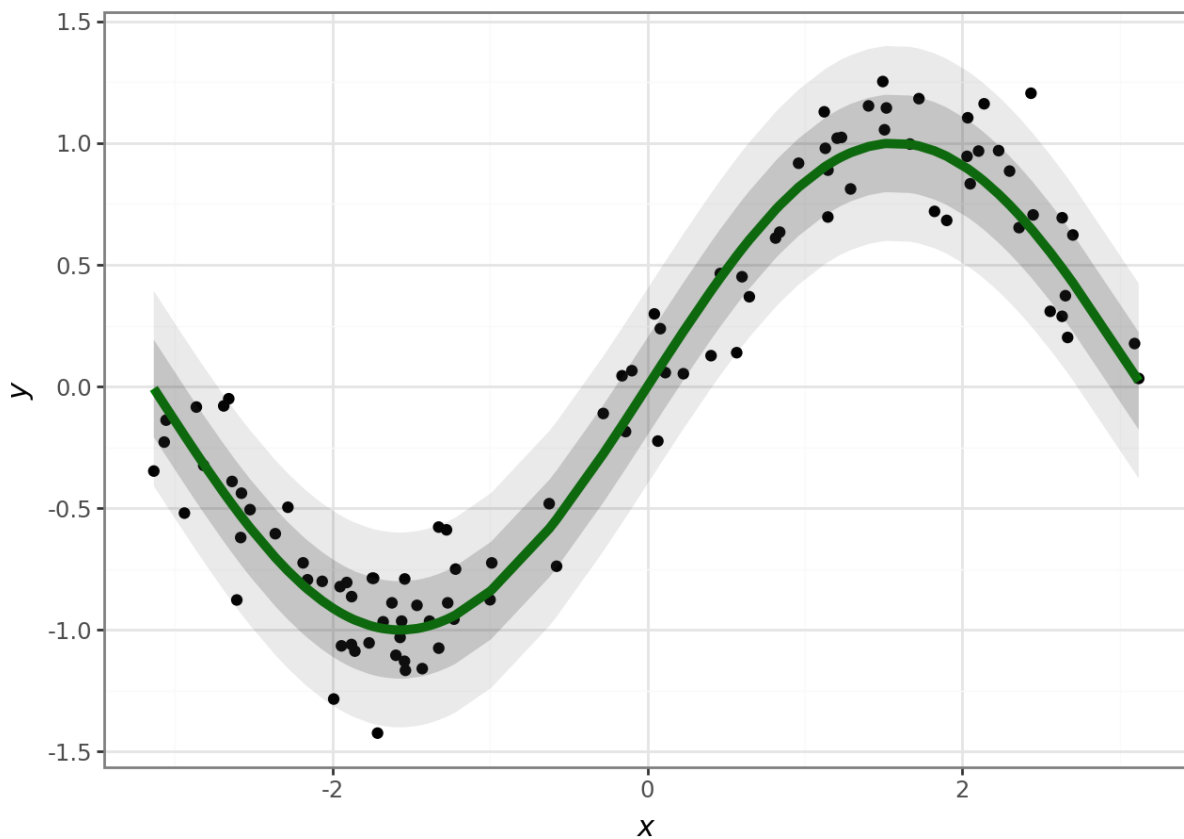
First, I read the given file in google colab, and ran all blocks of the given file without any modifications. The following figures are the results of the run.

```
✓ 12 주
[1] 1 # @title Imports
    2
    3 import numpy as np
    4 import pandas as pd
    5 import plotnine as gg
    6 import sklearn
    7
    8 gg.theme_set(gg.theme_bw());

✓ 0 주
[2] 1 # @title Create the dataset
    2
    3 num_data = 100
    4 noise_sigma = 0.2
    5
    6 train_xs = np.random.uniform(-np.pi, np.pi, size=num_data)
    7 train_ys = np.sin(train_xs) + noise_sigma * np.random.randn(num_data)

✓ 3 주
[3] 1 # @title Plot the data and its underlying distribution.
    2
    3 df = pd.DataFrame({'x': train_xs,
    4                   'y': train_ys,
    5                   'h': np.sin(train_xs),
    6                   'std': noise_sigma,
    7                   })
    8
    9 p = (gg.ggplot(df)
    10     + gg.aes(x='x', y='h')
    11     + gg.geom_point(gg.aes(y='y'))
    12     + gg.geom_line(color='green', size=2)
    13     + gg.geom_ribbon(gg.aes(ymin='h - std', ymax='h + std'), alpha=0.2)
    14     + gg.geom_ribbon(gg.aes(ymin='h - 2 * std', ymax='h + 2 * std'), alpha=0.1)
    15     + gg.ggtitle(r'$p(x, y)$: Mean in green, $\sigma$ '
    16                 'and $2\sigma$ intervals in grey')
    17     + gg.xlab(r'$x$')
    18     + gg.ylab(r'$y$')
    19 )
    20 p
```

$p(x, y)$: Mean in green, σ and 2σ intervals in grey

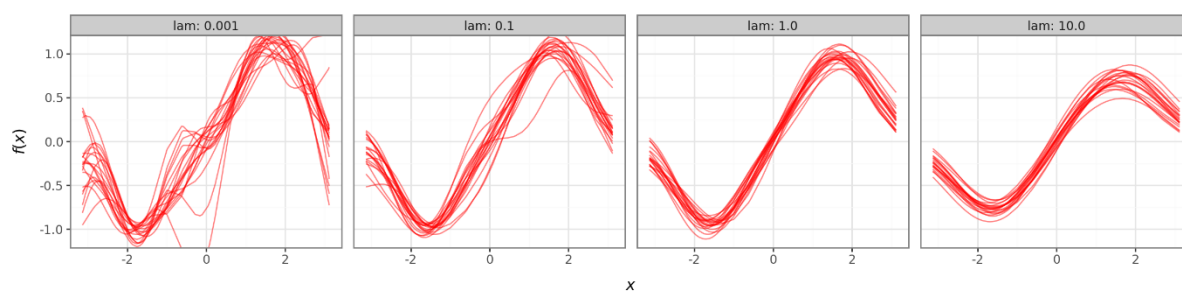


```
[4] 1 # Number of training points, number of datasets to sample, number of features to use.
2 num_train = 25
3 num_models = 20
4 num_features = 24
5
6 # Define the features.
7 polynomial_features = np.array([train_xs**n for n in range(num_features)]).T
8 fourier_features = np.array([np.sin(2*np.pi*n*train_xs / num_features) for n in range(num_features)]).T
9 gaussian_features = np.array([np.exp(-(train_xs - (2*np.pi*x/num_features - np.pi))**2) for x in range(num_features)]).T
10
11 # Train for different amounts of regularization.
12 features = gaussian_features
13 results_df = pd.DataFrame()
14 for lam in [1e-3, 1e-1, 1e0, 1e1]:
15     for idx in range(num_models):
16         # Sample a small subset of the dataset.
17         train_idx = np.random.randint(len(features), size=num_train)
18         train_features = features[train_idx]
19         train_targets = train_ys[train_idx]
20
21         # Fit a model to it.
22         model = sklearn.linear_model.Ridge(alpha=lam)
23         model.fit(train_features, train_targets)
24
25     # Collect results.
26     df = pd.DataFrame({
27         'x': train_xs,
28         'y': train_ys,
29         'h': np.sin(train_xs),
30         'f': model.predict(features),
31         'idx': [idx] * num_data,
32         'lam': lam,
33     })
34     results_df = pd.concat([results_df, df])
```

```

✓ 3本 [5] 1 # Plot our fitted functions.
2 fit_plot = (gg.ggplot(results_df)
3             + gg.aes(x='x')
4
5             + gg.geom_line(gg.aes(y='f', group='idx'), color='red', alpha=0.5)
6             + gg.facet_wrap('lam', labeller='label_both', ncol=4)
7             + gg.coord_cartesian(ylim=[-1.1, 1.1])
8             + gg.theme(figure_size=(12, 3))
9             + gg.xlab(r'$x$')
10            + gg.ylab(r'$f(x)$')
11 )
12 fit_plot

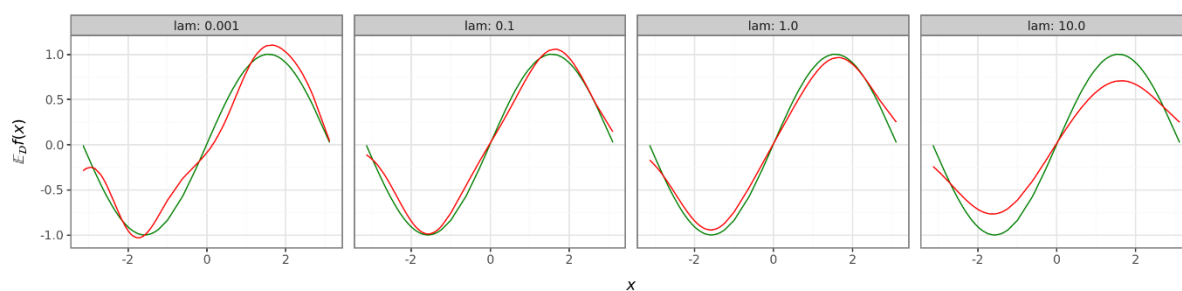
```



```

✓ 0本 [6] 1 # Plot their mean, over the datasets D.
2 mean_df = results_df.groupby(['x', 'lam', 'y', 'h']).agg({'f': "mean"}).reset_index()
3
4 mean_plot = (gg.ggplot(mean_df)
5              + gg.aes(x='x', y='f')
6              + gg.geom_line(gg.aes(y='h'), color='green')
7              + gg.geom_line(color='red')
8              + gg.facet_wrap('lam', labeller='label_both', ncol=4)
9              + gg.theme(figure_size=(12, 3))
10             + gg.xlab(r'$x$')
11             + gg.ylab(r'$\mathbb{E}_{\mathcal{D}}f(x)$')
12 )
13 mean_plot

```



[Task 2]

I added five different distribution functions, $p(x)$ s, in the 'Create the dataset' part of the given code. In the following figure, the 0th $p(x)$ is uniform distribution function which is the default one given in the original file. The 1st, 2nd, 3rd $p(x)$ s are exponential, gamma, logistic function each. And for the 4th and 5th, they are Gaussian functions with standard deviation 1 and 1.5 respectively. Among those six $p(x)$ s, I will find out which one is the most suitable for the ground-truth model.

Theoretically, a Gaussian distribution function is the least assumptive and can express most situations from the real world. Thus, I expect that the 4th or 5th $p(x)$ s, the Gaussian functions, will show the best results for being ground-truth.

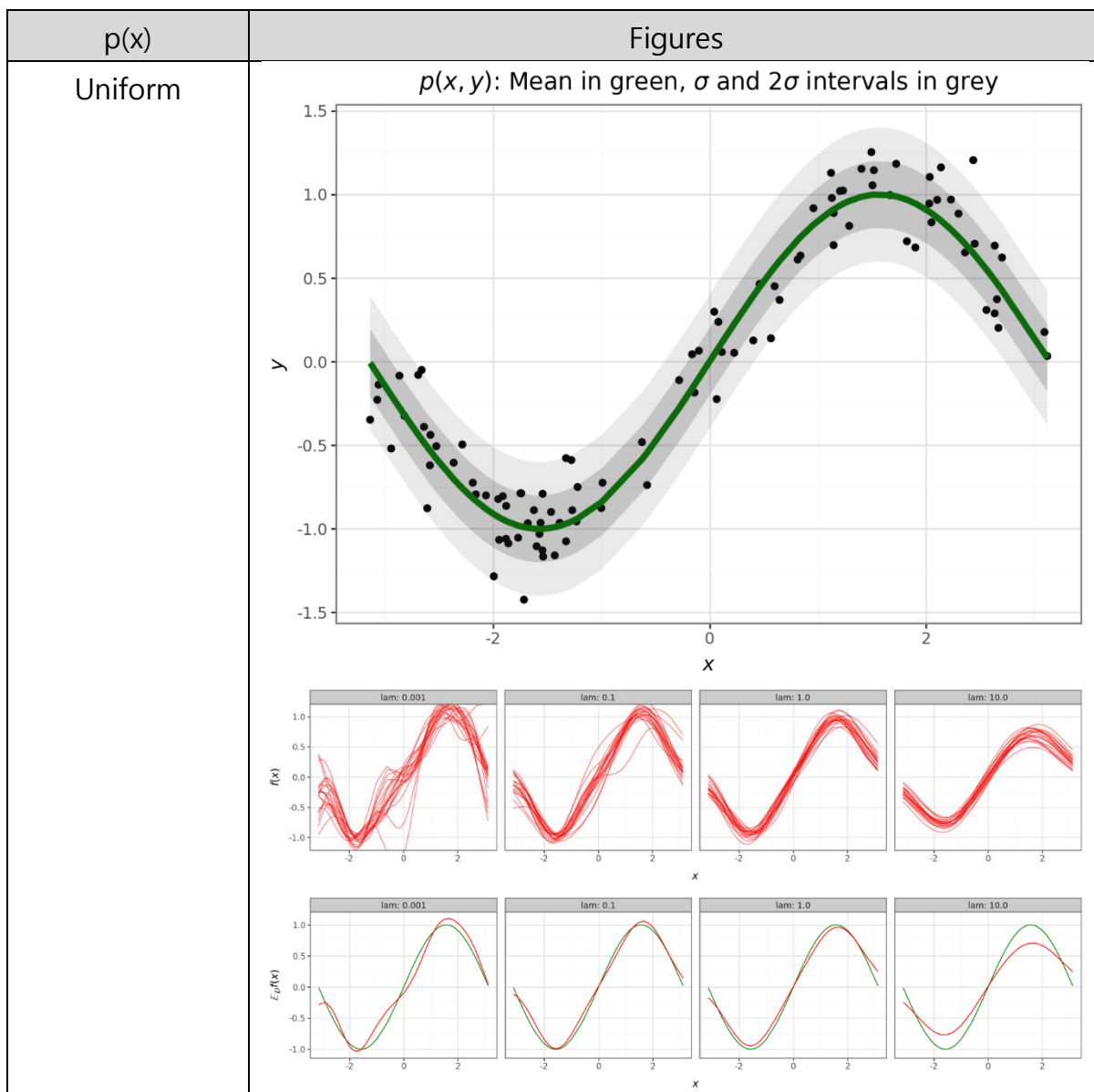
```
▼ Create the dataset

1 # @title Create the dataset
2
3 num_data = 100
4 noise_sigma = 0.2
5
6 # p(x) 0: Uniform
7 train_xs = np.random.uniform(-np.pi, np.pi, size=num_data)
8
9 # p(x) 1: Exponential
10 train_xs = np.random.exponential(1, size=num_data)
11
12 # p(x) 2: Gamma
13 train_xs = np.random.gamma(2, size=num_data)
14
15 # p(x) 3: Logistic
16 train_xs = np.random.logistic(size=num_data)
17
18 # p(x) 4: Gaussian(Normal) with scale 1
19 train_xs = np.random.normal(0, 1, size=num_data)
20
21 # p(x) 5: Gaussian(Normal) with scale 1.5
22 train_xs = np.random.normal(0, 1.5, size=num_data)
23
24 train_ys = np.sin(train_xs) + noise_sigma * np.random.randn(num_data)
```

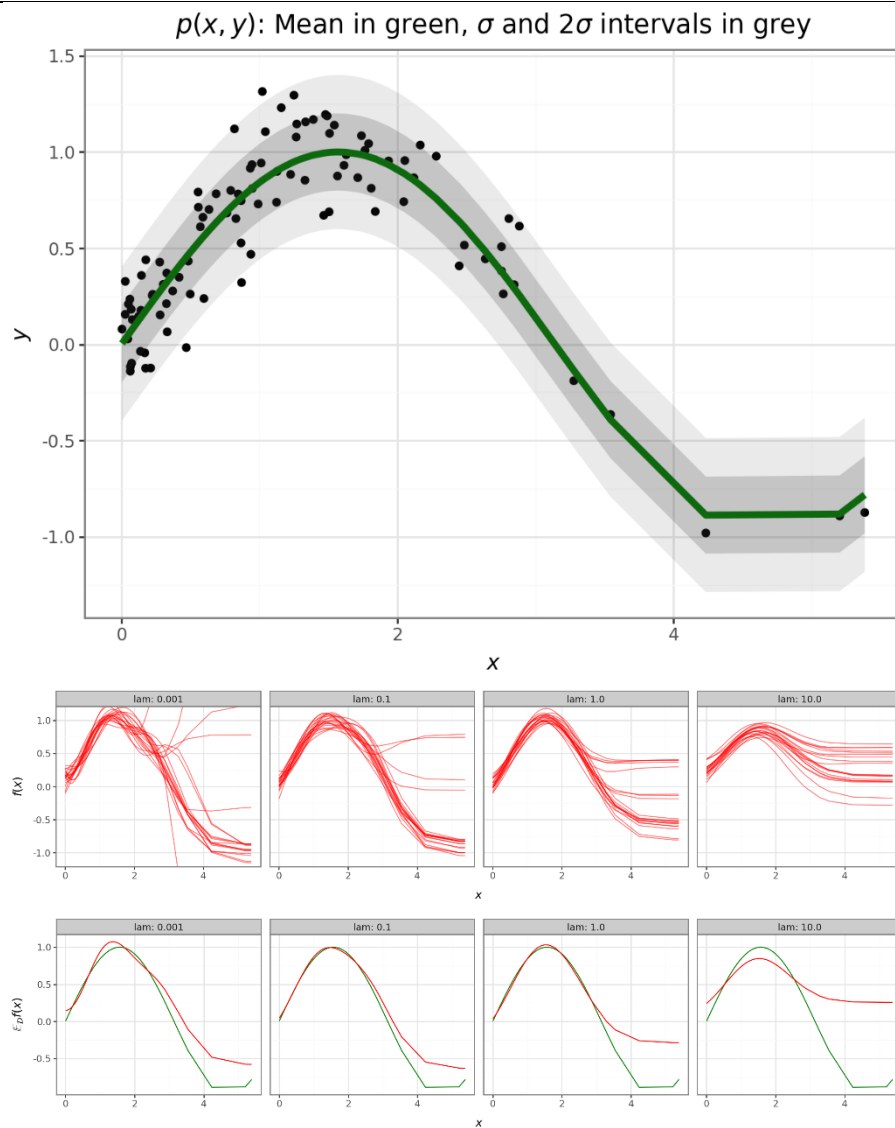
▼ Create the dataset

```
1 # @title Create the dataset
2
3 num_data = 100
4 noise_sigma = 0.2
5
6 # # p(x) 0: Uniform
7 # train_xs = np.random.uniform(-np.pi, np.pi, size=num_data)
8
9 # # p(x) 1: Exponential
10 train_xs = np.random.exponential(1, size=num_data)
11
12 # # p(x) 2: Gamma
13 # train_xs = np.random.gamma(2, size=num_data)
14
15 # # p(x) 3: Logistic
16 # train_xs = np.random.logistic(size=num_data)
17
18 # # p(x) 4: Gaussian(Normal) with scale 1
19 # train_xs = np.random.normal(0, 1, size=num_data)
20
21 # # p(x) 5: Gaussian(Normal) with scale 1.5
22 # train_xs = np.random.normal(0, 1.5, size=num_data)
23
24 train_ys = np.sin(train_xs) + noise_sigma * np.random.randn(num_data)
```

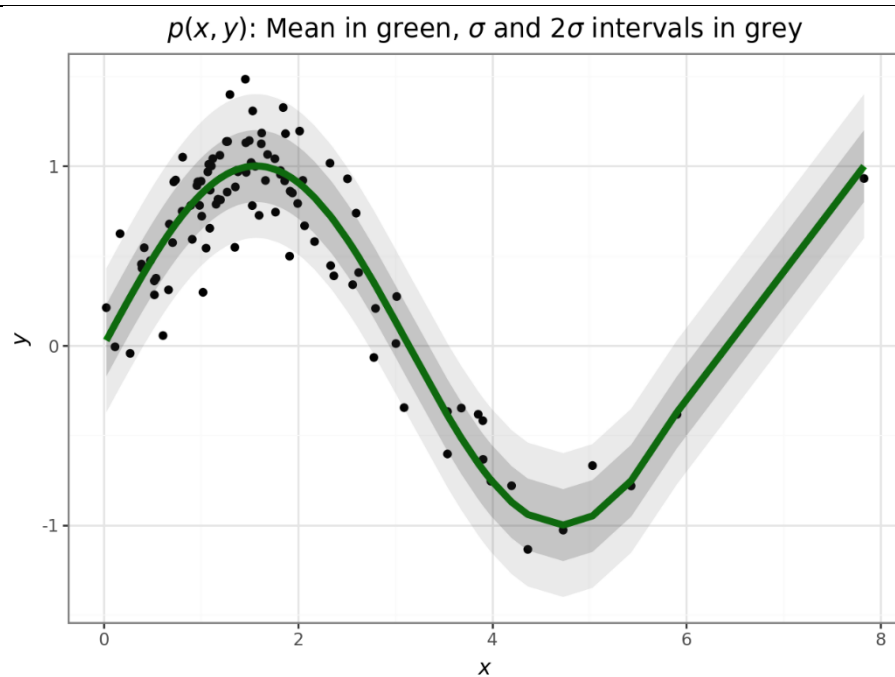
To do the experiment, for each $p(x)$, I activated the function by making all the other lines comment, and ran the whole code. The figures obtained from each run are as follows.

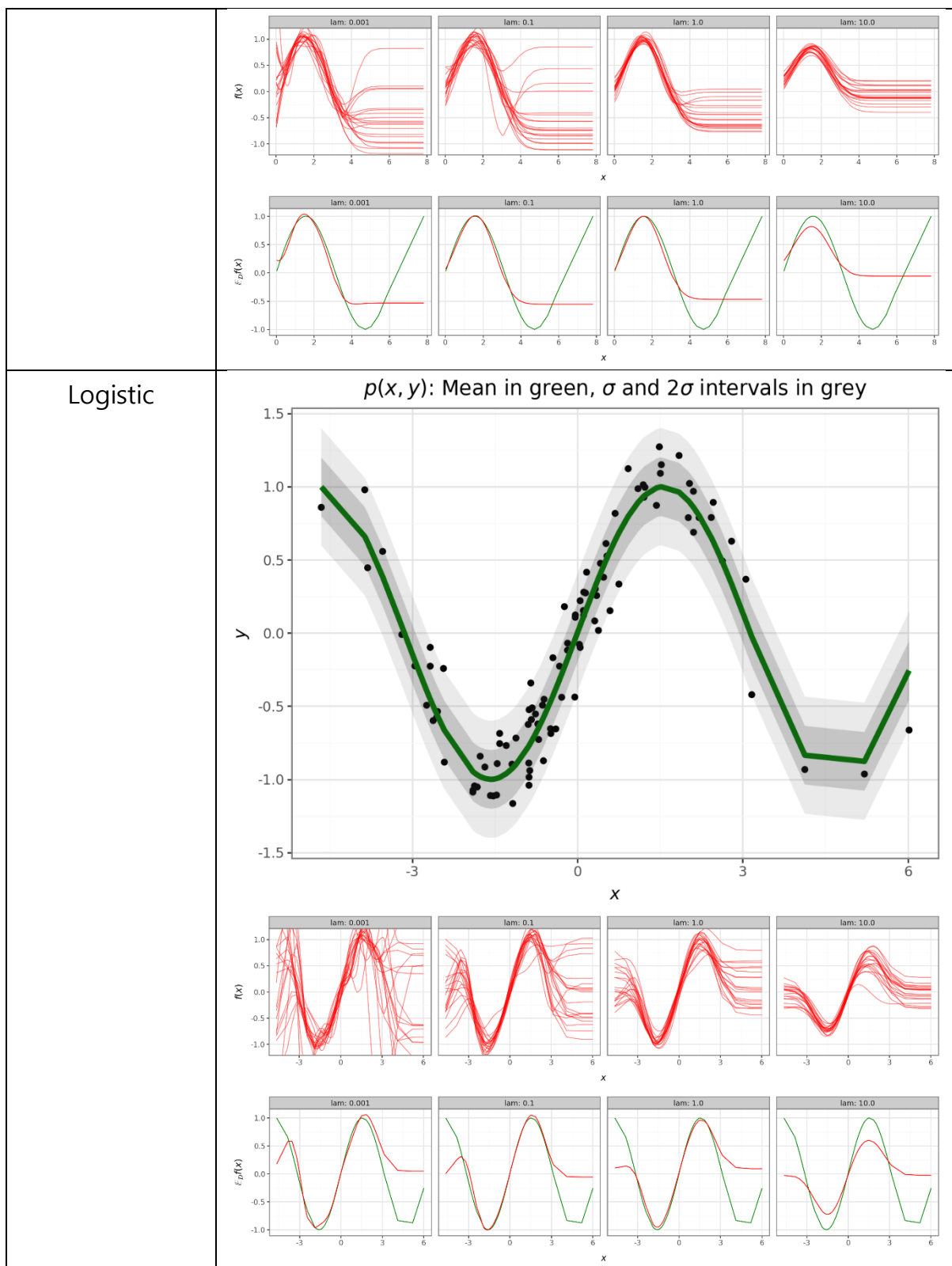


Exponential

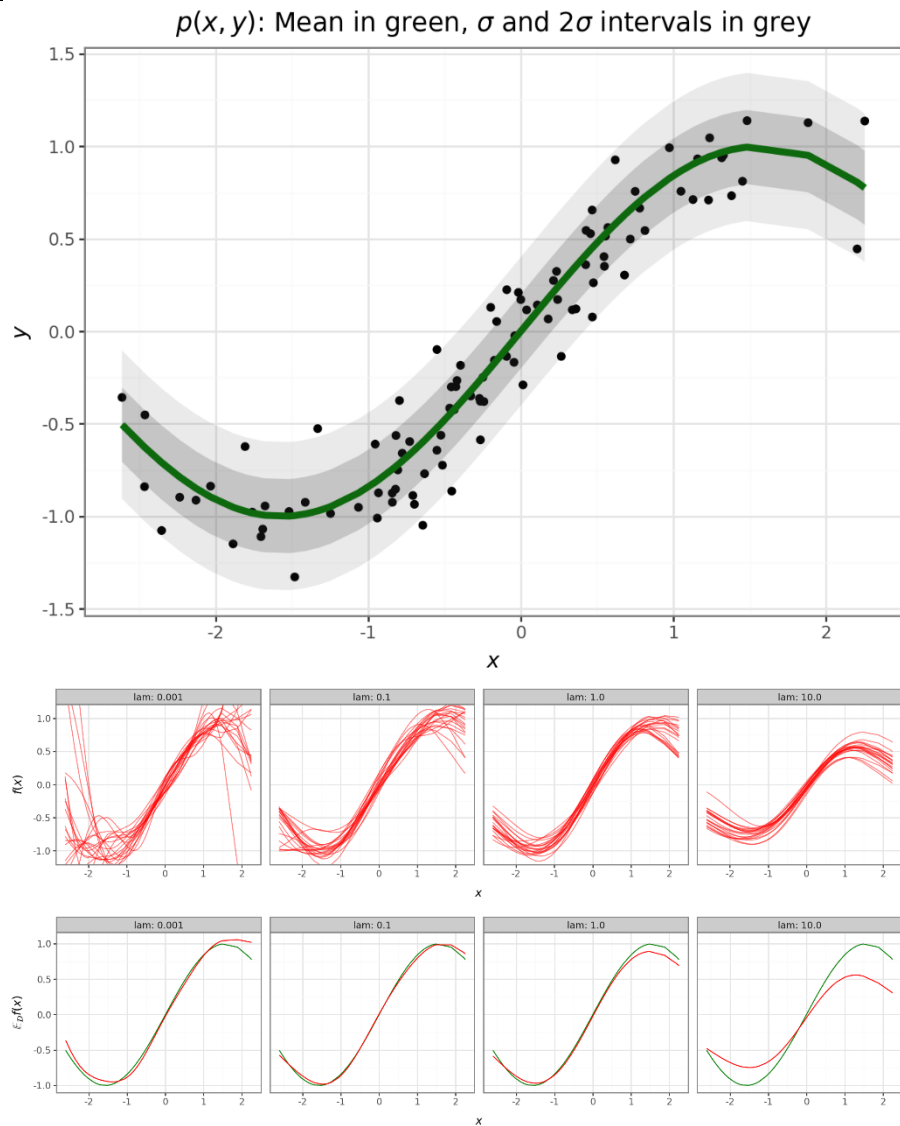


Gamma

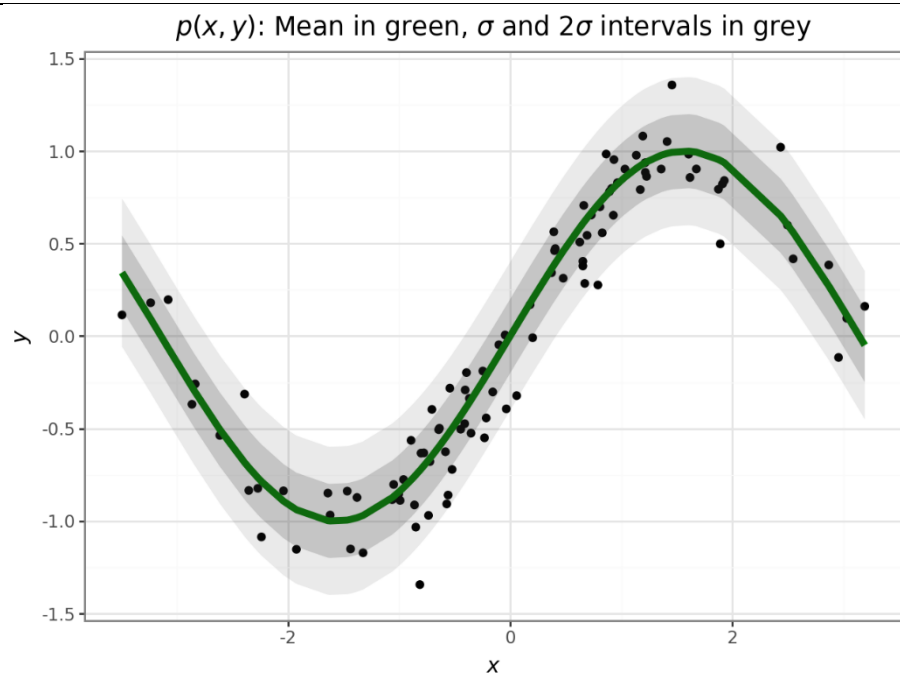


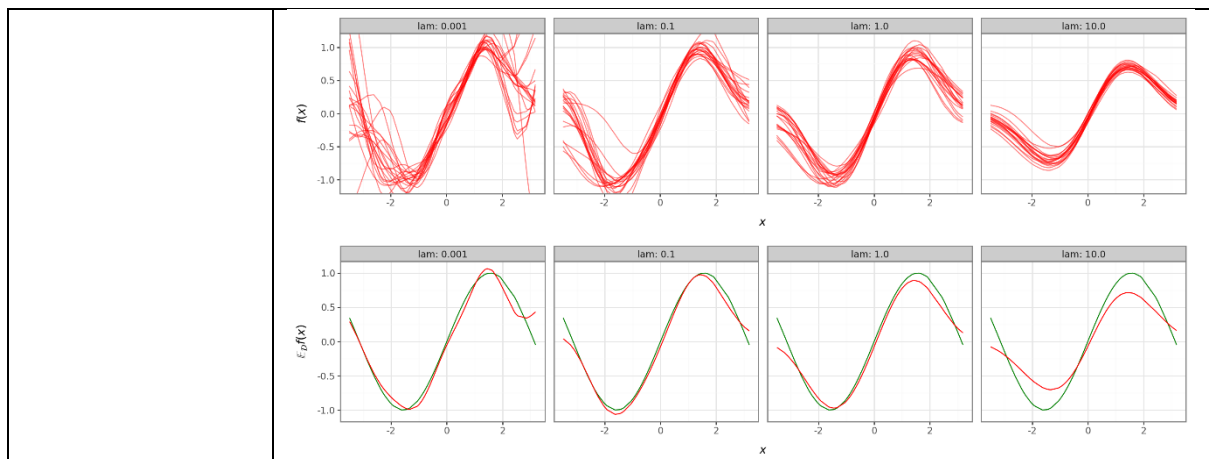


Gaussian with
scale 1



Gaussian with
scale 1.5





As expected, it seems that the graph shows the best result when $p(x)$ is Gaussian with scale 1, and I will choose it as $p(x)$ for next tasks.

[Task 3]

Create the dataset

```

1 # @title Create the dataset
2
3 num_total_data = 1000
4 noise_sigma = 0.2
5
6 np.random.seed(0)
7
8 data_splits_ratios = [
9     (0.9, 0.05, 0.05),
10    (0.8, 0.1, 0.1),
11    (0.7, 0.15, 0.15),
12    (0.6, 0.2, 0.2),
13    (0.5, 0.25, 0.25)
14 ]
15 data_splits_ratio = data_splits_ratios[0]
16
17 num_train_data = int(num_total_data * data_splits_ratio[0])
18 num_val_data = int(num_total_data * data_splits_ratio[1])
19 num_test_data = int(num_total_data * data_splits_ratio[2])
20
21 train_xs = np.random.normal(0, 1, size=num_train_data)
22 val_xs = np.random.normal(0, 1, size=num_val_data)
23 test_xs = np.random.normal(0, 1, size=num_test_data)
24
25 train_ys = np.sin(train_xs) + noise_sigma * np.random.randn(num_train_data)
26 val_ys = np.sin(val_xs) + noise_sigma * np.random.randn(num_val_data)
27 test_ys = np.sin(test_xs) + noise_sigma * np.random.randn(num_test_data)

```

To do both tasks 3-1 and 3-2 at once, I introduced several new variables. 'num_total_data' is the sum of the numbers of train, validation, and test data, which is set as 1000. Also, to make the same result in every run, I set np.random.seed as 0. 'data_splits_ratios' contains five different ratio of train, validation, and test data to 'num_total_data', and 'num_train[/val/test]_data' means the actual number of train[/val/test] data. To extract a certain ratio from the list 'data_splits_ratios', I implements a line 'data_splits_ratio = data_splits_ratios[0]'. I changed it directly from 'data_splits_ratios[0]' to 'data_splits_ratios[4]' in each run. For instance, the tuple (0.8, 0.1, 0.1) represents 800 train data, 100 validation data, and 100 test data among 1000 data. Using those variables, I generated three independent pairs of (xs, ys) where ys is the ground-truth values on xs.

```
1 # Number of training points, number of datasets to sample, number of features to use.
2 num_train = 25
3 num_models = 1
4 num_features = 24
5
6 # Define the features.
7 polynomial_features = np.array([train_xs**n for n in range(num_features)]).T
8 fourier_features = np.array([np.sin(2*np.pi*n*train_xs / num_features) for n in range(num_features)]).T
9 gaussian_features = np.array([np.exp(-(train_xs - (2*np.pi*x/num_features - np.pi))**2) for x in range(num_features)]).T
10
11 train_features = np.array([np.exp(-(train_xs - (2*np.pi*x/num_features - np.pi))**2) for x in range(num_features)]).T
12 val_features = np.array([np.exp(-(val_xs - (2*np.pi*x/num_features - np.pi))**2) for x in range(num_features)]).T
13 test_features = np.array([np.exp(-(test_xs - (2*np.pi*x/num_features - np.pi))**2) for x in range(num_features)]).T
14
15 train_targets = train_ys
16
17 # Train for different amounts of regularization.
18 features = gaussian_features
19 results_df = pd.DataFrame()
20
21 best_val_loss = 10000
22 best_lam = 0
23 best_model = None
24
```

```

25 for lam in [1e-3, 1e-1, 1e0, 1e1]:
26     for idx in range(num_models):
27         # Sample a small subset of the dataset.
28         # train_idx = np.random.randint(len(features), size=num_train)
29         # train_features = features[train_idx]
30         # train_targets = train_ys[train_idx]
31
32         # Fit a model to it.
33         model = sklearn.linear_model.Ridge(alpha=lam)
34         model.fit(train_features, train_targets)
35
36         val_pred = model.predict(val_features)
37         val_loss = mean_squared_error(val_ys, val_pred)
38
39         if val_loss < best_val_loss:
40             best_val_loss = val_loss
41             best_lam = lam
42             best_model = model
43
44         # Collect results.
45         df = pd.DataFrame({
46             'x': train_xs,
47             'y': train_ys,
48             'h': np.sin(train_xs),
49             'f': model.predict(train_features),
50             'idx': [idx] * len(train_xs),
51             'lam': lam,
52             'val_loss': val_loss
53         })
54         results_df = pd.concat([results_df, df])

```

```

55
56 print(f'Best lambda: {best_lam}')
57 print(f'Best validation loss: {best_val_loss}')
58
59 test_pred = best_model.predict(test_features)
60 test_loss = mean_squared_error(test_ys, test_pred)
61 print(f'Test loss: {test_loss}')

```

I don't need to subsample the train set, so I set 'num_models' as 1. Since I selected Gaussian features for data, I introduced three new features variables, 'train_features', 'val_features', and 'test_features'. For various lamda(1e-3, 1e-1, 1, 10), I trained each model with train features, and calculated 'val_loss'(validation loss) with 'mean_squared_error' between 'val_ys' and 'val_pred'. After four iterations, I used the best model, which has the lowest validation loss, to predict 'test_pred' with 'test_xs' and compared it to 'test_ys'. The following chart shows the best lambda for the best validation loss and test loss for each 'data_splits_ratio'.

data_splits_ratio (num_total_data = 1000)	Best lamda Best validation loss Test loss
(0.9, 0.05, 0.05)	Best lambda: 0.1 Best validation loss: 0.03693551023278932 Test loss: 0.03711759277891138

(0.8, 0.1, 0.1)	Best lambda: 10.0 Best validation loss: 0.03288240470670067 Test loss: 0.03673891779039666
(0.7, 0.15, 0.15)	Best lambda: 10.0 Best validation loss: 0.04089810467330293 Test loss: 0.03424990421899294
(0.6, 0.2, 0.2)	Best lambda: 1.0 Best validation loss: 0.040728039736796146 Test loss: 0.03505408538852402
(0.5, 0.25, 0.25)	Best lambda: 0.1 Best validation loss: 0.04047126478061548 Test loss: 0.035814433051663895

We can get the lowest test loss(mean squared error between 'test_pred' and 'test_ys') when 'data_splits_ratio' is (0.7, 0.15, 0.15) from the model with lambda 10.

[Task 4]

Regularization is a technique used to prevent overfitting by adding a penalty term on a model's weights to the loss function. The most two common methods are L1 and L2 regularizations, which penalizes the sum of absolute values of weights or the sum of squared values of weights each. In this homework, we used Ridge model which has L2 regularization term in its loss function. Since the term penalizes large weights, it makes the model less sensitive to specific data points or an outlier so that it prevents overfitting. In regularization term, there is a hyperparameter, lambda, which controls influence of the term. If lambda is too small, that is, when the regularization term has little impact on the loss function of a model, the model may overfit to the training data. Otherwise, with large regularization term, the model may definitely be far from overfitting but may show low accuracy. Thus, we have to find out proper lambda for a model.

Validation is the process of selecting the best model by evaluating different hyperparameters on a separate validation set, which is distinct from training set and test set. Before the final testing, which means testing the model with test set, we check the performance of the model with validation set and estimate how well the model generalizes to unseen data. By testing different validation sets with various hyperparameters settings, we can determine the best configuration

for our model. In this homework, we choose the best model with the best λ which minimizes 'mean squared error' on the validation set and apply it on test set. Thus, we can improve the generalization of the model with validation.

Bias-variance decomposition explains the tradeoff between underfitting and overfitting. It is hard to satisfy both small bias and small variance at the same time, that is, we should find an appropriate balance between bias and variance through validation. Let's assume that we are going to train multiple models on different subsamples of the original data. Then, we can analyze bias and variance across these models. When bias is high, a model cannot capture essential patterns in data well, leading to high training and high validation errors, so it becomes underfitted. Otherwise, when variance is high, it means that the model is highly complex and has learned from training data too much so that it results in low training error but high validation error, that is, it overfits.

To summarize, regularization, validation, and bias-variance decomposition are the main concepts to improve the performance of a model and to generalize it for any type of data. We can achieve optimal generalization and avoid underfitting or overfitting by adjusting the complexity of a model using those techniques. To minimize the loss function, which includes regularization term and both bias-variance components, and evaluating multiple models on the validation sets, we can select the best model with the most suitable hyperparameters.

[Task 5]

- (i) A hyperparameter means a parameter which is set before the training begins and is not learned from training data. It controls the behavior of a model and influences its performance. Unlike model parameters such as weights in neural networks, hyperparameters are determined explicitly and must be tuned by humans using some techniques like validation. The following are some examples of hyperparameters: learning rate, regularization strength (i.e. λ in this homework), batch size, the number of hidden layers in neural networks, dropout rate, and so on.

- (ii) The larger a model, the more the number of parameters in the model. The more the number of parameters, the more complex the pattern that the model learns. Since ChatGPT is LLM(Large Language Model), it has many parameters and it is likely to be overfitted. If it has too many parameters, then it can learn highly complex patterns from training data and may overfit, that is, variance increases but bias decreases. On the other hand, if ChatGPT does not have enough parameters, then the smaller model generalizes better but shows low accuracy, that is, bias increases whereas variance decreases. Thus, OpenAI should fine-tune the number of parameters in ChatGPT to balance complexity and generalization. Also, there are some techniques to control bias and variance of ChatGPT other than the number of parameters in it. Techniques such as regularization, dropout rate, or weight decay makes the impact of weights weaker, so it prevents overfitting by reducing variance. And data augmentation hinders the model learning on specific data, so we can avoid overfitting and the model generalizes.